Open-Source Agentic Plugin Stack

A ranked stack, credibility signals, and a practical blueprint for building a ChatGPT-style UI with real tools via MCP

Prepared 2026-03-03

Executive summary

You asked for the real, usable plugin stack - not a fluffy list - that lets any user operate through a ChatGPT-like interface (chat, image gen, web search, market monitoring, document workflows) and also kick off autonomous CLI execution for real work (code changes, tests, PRs, CI). The only way to do that with an open-source, free stack is MCP-first: tools are external, standardized, auditable, and permissionable.

This document does three things:
1) Defines what "plugins" actually mean in 2026 (MCP servers + tool bridges + guardrails + evals).
2) Ranks the minimum set of components required to reliably "create anything" without turning into an un-debuggable toy.
3) Gives you an implementation blueprint that a senior engineer can build, secure, and operate.

Important reality check: software can be open-source and "free", but compute is never free. You either pay in hardware (GPUs, RAM, energy) or in API spend. This thesis focuses on free/open licensing and self-hostable infrastructure, while being explicit about compute trade-offs.

How to read this

- Section 2 gives a strict definition of 'plugin' and the failure modes that kill agent stacks.
- Section 3 is the ranked stack (what you need first, and why).
- Section 4 is how to wire it end-to-end (UI -> orchestration -> tools -> CI -> audit).
- Sections 5–7 cover security, reliability, cost/compute, and real-world scenarios.
- Appendices give discovery loops so your stack stays fresh without chasing hype.

1. What 'plugin' means now

Stop thinking 'plugin' as UI extensions like old IDE marketplaces. In agent systems, "plugins" are mostly tool interfaces with three layers:

A) Tool contract: a structured schema the model can call (inputs/outputs), plus transport (HTTP/stdio/SSE).
B) Execution boundary: where the tool runs, what it can access, and what happens if it misbehaves.
C) Policy boundary: who is allowed to call it, with what data, and with what side effects.

MCP (Model Context Protocol) is the current convergence point for A+B: it standardizes how LLM apps connect to external tools and data sources [R1]. MCP servers act as the plugin surface area, and you can publish/discover them via registries/lists (MCP Registry [R3], community lists like awesome-mcp-servers [R4], and community chatter [R43]).

For your requirement ("any user can create anything"): the stack must cover:
- interactive UI (chat + artifacts + file uploads),
- tool connectors (filesystem, git, browser automation, web search, DBs),
- image generation,

- autonomous coding/execution flows,
- evaluations, tracing, and security gates.

If you skip evals + tracing + security gates, you do not have a production stack; you have a demo.

The 7 failure modes that kill agent stacks

- 1) Tool sprawl: 200 tools, no permissions model, no audit trail.
- 2) Non-deterministic behavior: no evals; upgrades break workflows silently.
- 3) Unsafe execution: tools can run arbitrary shell / exfiltrate keys (stdio bridges and misconfigs are common traps).
- 4) Context poisoning: web search and docs ingestion pollute decisions without attribution.
- 5) No observability: you can't replay failures or explain costs (token, time, GPU).
- 6) No CI gates: autonomous changes merge without security and regression checks.
- 7) UX mismatch: dev tools are exposed to non-dev users without guardrails, leading to chaos.

2. Ranking rubric: how this stack is scored

You asked for credibility. So ranking is not vibes - it is a rubric. Each component is scored 1–5 across five dimensions (25 max):

1) Adoption: stars/forks, community usage, ecosystem presence (GitHub + Reddit + social signals).
2) Maturity: stability, release cadence, docs quality, multi-user readiness.
3) Integration: how easily it plugs into MCP-first architecture without glue-code hell.
4) Security posture: known issues, sandboxing options, permission model, incident response.
5) Operator cost: complexity to deploy/upgrade, resource requirements, day-2 operations.

The rankings below are opinionated, but anchored to public signals and documented capabilities (see References).

3. Ranked stack

This is the stack in the order you should adopt it. Earlier items are foundations; later items are multipliers.

Rank	Component	Category	Why it matters (in practice)	Primary use	Evidence
1	MCP protocol + SDKs	Tool contract standard	Avoid N×M integrations. Make tools portable across UIs/agents.	Standard tool calling	R1, R2
2	MCP discovery	Find/curate	You need a	Discover tools	R3, R4, R43

	(Registry + awesome lists)	tools	catalog or you reinvent everything.		
3	Chat UI (LibreChat)	Multi-user front door	MCP support + agents + artifacts + multi-user auth in one place.	User interaction	R9, R10
4	Alt Chat UI (Open WebUI)	Simpler offline UX	Mass adoption; strong offline UX; strong tool story; huge repo traction.	User interaction	R5, R6, R42
5	MCP bridge (mcpo) + OpenAPI tool servers	Compatibility layer	Practical way to expose MCP tools over OpenAPI; helps web UIs.	Tool bridge	R7, R8
6	Local model runtime (Ollama)	Free inference option	Run open models locally; standard endpoint for your UIs/agents.	Inference runtime	R12
7	Web search backend (SearXNG)	Real-time search	Self-hosted metasearch to avoid paid APIs; privacy friendly.	Search	R13
8	Browser automation (Playwright wrapped as a tool)	Do things on websites	Agents that can't browse and click can't actually execute workflows.	Web actions	R2
9	Image generation (ComfyUI + Manager)	Chat-to-image	Best open workflow engine for diffusion pipelines, extensible nodes.	Image gen	R14, R15
10	Orchestration core (LangGraph)	State + retries + memory	Graph state prevents spaghetti agents; supports long-running	Workflow state	R18

			flows.		
11	Multi-agent framework (CrewAI)	Role-based subagents	Simple multi-agent patterns when you want PM/Engineer/QA separation.	Sub-agent teams	R19
12	Alt multi-agent framework (AutoGen)	Conversational multi-agent	Good research/prototyping patterns; large ecosystem.	Sub-agent teams	R20
13	Autonomous coding agent (OpenHands)	Repo-edit + run	An actual coding agent platform with CLI and agent SDK.	Repo automation	R21
14	Terminal coding assistant (Aider)	Diff-based edits	Fast, git-native editing with clean diffs and commits.	Repo edits	R22
15	IDE agent (Cline)	High-control IDE workflow	Mass adoption; permissioned execution in IDE; MCP patterns.	IDE automation	R23
16	CI agent checks (Continue)	Quality gates	AI checks as status checks. Stops agent regression from shipping.	CI quality gates	R24
17	Observability (Langfuse)	Traces + metrics	Without traces you can't debug multi-step failures or costs.	Tracing/metrics	R16
18	Evals + red teaming (promptfoo)	Regression tests	CI-native evals and security testing for prompts/agents/RAG.	Regression/security	R17
19	Direction file standard (AGENTS.md)	Stop wandering	Single place to instruct agents; reduces tool-specific clutter.	Agent alignment	R25
20	Tone/style	Tone	Make tone a	Tone	R26

	linting (Vale + styles)	management	build artifact: lint docs, prompts, outputs.	enforcement	
21	Prose linting (Proselint)	Clarity lint	Cleaner writing; catches common prose issues.	Clarity	R27
22	Inclusive language (alex)	Avoid landmines	Flags problematic language and suggests alternatives.	Safety/brand	R28
23	PII redaction (Presidio)	Data hygiene	Detect/redact PII before logs/prompts leave the boundary.	Privacy	R29
24	Programmable rails (NeMo Guardrails / Guardrails AI)	Policy enforcement	Rules for what can be said/done; structured output validation.	Policy	R30, R31
25	Secrets & AppSec gates (Gitleaks, OSV, Trivy, Opengrep)	Security gates	Prevent keys, vuln deps, container issues, and obvious code flaws.	Security gates	R32, R33, R34, R35
26	Search + retrieval (Meilisearch + Qdrant)	Docs + memory	Hybrid search + vector retrieval for document-heavy workflows.	RAG/search	R36, R37
27	Market monitoring (changedetection.io)	Price/restock alerts	Turns 'market search' into a repeatable signal pipeline.	Monitoring	R38
28	Automation agents (Huginn)	IFTTT on your server	General event agents; useful for always-on monitoring + routing.	Event automation	R39
29	Docs system	Documentation	You need	Docs/runbooks	R40, R41

(MkDocs Material / Docusaurus)	factory	runbooks, traces, and user-facing docs to scale usage.

What I would build first (the minimum viable 'create anything' stack)

- LibreChat (UI) [R9] or Open WebUI (simpler) [R5]. Pick one; don't run both unless you have a reason.
- MCP + a handful of core MCP servers (filesystem, git, browser automation) [R1][R2].
- mcpo as a bridge when your UI expects OpenAPI tool servers [R7][R8].
- Ollama for local inference (free licensing; your hardware does the work) [R12].
- SearXNG for web search [R13] + Playwright-based browsing as a callable tool [R2].
- ComfyUI for image generation workflows [R14][R15].
- Langfuse for traces [R16] and promptfoo for evals [R17] from day 1.
- AGENTS.md in every repo the agent touches [R25].
- Gitleaks + OSV + Trivy + Opengrep as non-negotiable CI gates [R32–R35].

4. Architecture blueprint

4.1 Reference architecture (MCP-first)

At a high level, you want the same shape every time:

User -> Chat UI -> Orchestrator -> Tools (MCP servers) -> Execution sandboxes -> CI gates -> Observability -> Audit logs

Key point: the UI should not have direct access to your production shell or secrets. Tools run behind controlled boundaries.

```
+--------------------+
|  Chat UI (LibreChat) |
|  or Open WebUI      |
+----------+----------+
         |
         | tool calls (MCP/OpenAPI)
         v
+--------------------+       +---------------------+
| Orchestrator       |<------>| Observability       |
| (LangGraph etc.)   |       | (Langfuse)          |
+----------+----------+       +--------------------- +
         |
         v
+--------------------+
| Tool Bus (MCP)     |
```

```
| - filesystem        |
| - git               |
| - browser (Playwright)
| - search (SearXNG)  |
| - image gen (ComfyUI)
+----------+----------+
          |
          v
+--------------------+       +---------------------+
| Sandboxed runners  |------>| CI Gates            |
| - ephemeral workspace|     | - gitleaks          |
| - container/K8s    |       | - osv-scanner       |
|                    |       | - trivy             |
|                    |       | - opengrep          |
+--------------------+       +---------------------+
```

4.2 Tool boundary rules (non-negotiable)

- All tools must be callable with explicit schemas (inputs/outputs). MCP gives you that [R1].
- Every tool call is logged (inputs redacted where needed) and trace-linked to the user session (Langfuse) [R16].
- Mutating tools (write file, open PR, deploy) require explicit user approval or a policy engine decision.
- Default to read-only tools. Make mutation an opt-in escalation.

4.3 Discovery and curation workflow (how you keep it 'reddit approved' without chasing hype)

You can’t manually vet 2000+ MCP servers. You need a repeatable curation loop:

Step 1: Pull candidates from the MCP Registry [R3], awesome lists [R4], and community chatter [R43].
Step 2: Score them (adoption, last commit, license, auth model, blast radius).
Step 3: Sandbox test: run the server behind a proxy (mcpo if needed) and run a standard test pack.
Step 4: Promote to "Approved" only after: (a) passes tests, (b) permissions model is clear, (c) logs are clean.
Step 5: Ship as a curated catalog visible in the UI (LibreChat supports MCP server management in UI releases [R10]).

5. Real-world scenarios (what this stack actually lets users do)

5.1 Research agent (web search + citations)

Goal: user asks a messy question; agent produces a sourced brief with links and a confidence score.

Required components:
- SearXNG for search [R13]
- Browser automation for follow-up reading (Playwright as tool) [R2]
- Langfuse traces for audit [R16]
- promptfoo evals to prevent regressions [R17]

Failure modes:
- Search engines block you; need engine configs and rate limits.
- Hallucinated citations; fix with 'source required' gates + eval tests.

5.2 Market monitoring agent (price/restock + alerts)

Goal: "watch these 30 SKUs and tell me when price drops or stock returns."

Required components:
- changedetection.io for monitoring and notifications [R38]
- Huginn for event routing/aggregation if you need multi-step automation [R39]
- Optional: browser automation for harder sites [R2]

Compute/cost:
- Mostly network + small CPU.
- Real cost is operational: sites break; you maintain selectors. Expect churn.

5.3 Design-to-implementation agent (image gen + UI build + tests)

Goal: user uploads a sketch or prompt; agent generates images, then scaffolds UI code, runs tests, opens a PR.

Required components:
- ComfyUI for image gen workflows [R14][R15]
- OpenHands / Aider / Cline for code changes [R21][R22][R23]
- Continue for PR checks [R24]
- Security gates (Gitleaks/OSV/Trivy/Opengrep) [R32–R35]

Reality:
- You will not get reliable autonomous merges without CI gates. Agents make subtle mistakes.

5.4 Document factory (policies, SOPs, onboarding, proposals)

Goal: produce consistent documents with stable tone, no PII leaks, and reliable structure.

Required components:
- AGENTS.md for repo instructions [R25]

- Vale + styles for tone consistency [R26]
- Proselint/alex for clarity and language checks [R27][R28]
- Presidio for PII detection/redaction [R29]
- MkDocs Material or Docusaurus to publish docs [R40][R41]

Outcome:
- Tone becomes testable, not subjective.

6. Security: stop your 'plugin stack' becoming an RCE stack

If you connect tools that can run commands, you are building a remote execution platform. Treat it like one.

Two public reminders:
- LibreChat has had an MCP stdio RCE advisory (patched quickly) [R11].
- Open WebUI has had a critical vulnerability involving Direct Connections (patch guidance exists) [R46].

The takeaway isn't 'avoid these projects'. It's: plan for tooling bugs and design blast-radius containment.

6.1 The permission model you actually need

- Read vs mutate split (default read-only).
- Per-user credentials: do not share a single PAT/API key across all users.
- Tenant isolation: tools should accept a tenant_id/user_id and enforce it.
- Short-lived tokens, minimal scopes, and revocation workflows.

6.2 Sandboxing patterns that work

- Run mutating tools in ephemeral workspaces (containers) with no long-lived secrets.
- Network egress allow-lists for tools: only allow domains you intend.
- Filesystem mounts: mount only the repo folder, not the host root.
- Audit logs: store tool call transcripts (with PII redaction).

6.3 CI security gates (mandatory)

If an agent can modify code, it must be forced through the same security gates as humans:

- Gitleaks: block leaked secrets [R32].
- OSV-Scanner: block known vulnerable dependencies [R33].
- Trivy: scan containers/IaC/filesystems [R34].
- Opengrep: fast SAST scanning for common patterns [R35].

7. Cost, compute, and efficiency

7.1 The real cost model

- Local inference (Ollama) shifts spend from tokens to hardware (GPU/RAM/time) [R12].
- Browser automation is I/O heavy and brittle; you pay in maintenance.
- Observability and evals add overhead, but they reduce fire drills and production failures (net savings).

7.2 Efficiency levers that matter

- Keep tools small and composable; avoid mega-tools with huge permissions.
- Cache expensive results (search, embeddings) where possible.
- Use evals to prevent prompt drift from causing reruns (promptfoo) [R17].
- Trace everything; debugging without traces is the biggest time sink (Langfuse) [R16].

8. Credibility signals (GitHub + Reddit + Instagram)

You asked for 'reddit approved' and hyped. Social proof is not correctness, but it is a signal for adoption and docs/community support.

Examples:
- Open WebUI shows very large GitHub traction (stars/forks visible on releases page) [R5].
- Cline shows strong traction (stars/forks visible on repo page) [R23].
- Reddit threads compare Open WebUI vs LibreChat and discuss MCP setup pain points [R42].
- Instagram has clear "local AI stack" hype around Open WebUI/Ollama/MCP - useful as a trend signal, not as technical validation [R44][R45].

Use social proof to decide what to test first, not what to trust blindly.

Appendix A: Your 'plugin catalog' sources (keep this updated)

- MCP Registry: publish/discover servers, app-store concept [R3].
- modelcontextprotocol/servers: reference implementations + patterns [R2].
- awesome-mcp-servers: broad community list (thousands) [R4].
- Reddit threads tracking 'most useful MCP servers' and ecosystem updates [R43].
- Open WebUI 'functions' repository: curated functions approved by core team [R5].

Appendix B: A practical 'Top 20 MCP tool categories' list

If you want 100+ tools without drowning: categorize first. These categories map cleanly to MCP servers and keep permissioning sane.

1. Filesystem (read/write scoped dirs)
2. Git (read, branch, commit, PR operations)
3. Repo hosting (GitHub/GitLab issues + PRs + reviews)

4. Browser automation (Playwright)
5. Web search (metasearch + SERP tools)
6. HTTP client (safe allow-listed fetch)
7. Databases (Postgres/MySQL/SQLite)
8. Data warehouses (ClickHouse, BigQuery etc)
9. Spreadsheets (CSV/XLSX read/write)
10. Docs & notes (Markdown vaults, wiki systems)
11. Email/calendar (OAuth, per-user tokens)
12. Chat ops (Slack/Discord bots)
13. Ticketing (Jira/Linear)
14. Cloud ops (K8s read-only; limited mutate)
15. Observability (Sentry/Logs)
16. Artifact store (S3-compatible buckets)
17. Image generation (ComfyUI workflows)
18. Audio/video pipelines (ffmpeg wrappers)
19. PDF/DOCX pipelines (conversion + extraction)
20. Notification routing (webhooks, NTFY, email)

Appendix C: References

All references were retrieved during web search. Use them as the authoritative link targets in your internal docs.

- R1 - Model Context Protocol (MCP) org + overview (GitHub) - https://github.com/modelcontextprotocol
- R2 - MCP servers reference implementations (GitHub) - https://github.com/modelcontextprotocol/servers
- R3 - MCP Registry (app-store concept) (GitHub) - https://github.com/modelcontextprotocol/registry
- R4 - Awesome MCP servers list (community discovery) (GitHub) - https://github.com/TensorBlock/awesome-mcp-servers
- R5 - Open WebUI (self-hosted chat UI) releases page showing stars/forks (GitHub) - https://github.com/open-webui/open-webui/releases
- R6 - Open WebUI docs (protocol-first + offline) overview (Docs) - https://docs.openwebui.com/
- R7 - Open WebUI mcpo (MCP-to-OpenAPI proxy) (GitHub) - https://github.com/open-webui/mcpo
- R8 - mcpo limitation: tools only (resources not yet supported) (GitHub) - https://github.com/open-webui/mcpo/discussions/94
- R9 - LibreChat repo (MCP support, agents, artifacts, multi-user) (GitHub) - https://github.com/danny-avila/LibreChat

- R10 - LibreChat release notes referencing dynamic MCP server management (GitHub) - https://github.com/danny-avila/LibreChat/releases
- R11 - LibreChat security advisory on MCP stdio RCE (patched) (GitHub) - https://github.com/danny-avila/LibreChat/security/advisories/GHSA-cxhj-j78r-p88f
- R12 - Ollama (local model runtime) repo (GitHub) - https://github.com/ollama/ollama
- R13 - SearXNG (self-hosted metasearch) repo (GitHub) - https://github.com/searxng/searxng
- R14 - ComfyUI (image gen workflows) repo (GitHub) - https://github.com/Comfy-Org/ComfyUI
- R15 - ComfyUI-Manager (extension management) (GitHub) - https://github.com/Comfy-Org/ComfyUI-Manager
- R16 - Langfuse (open-source LLM observability) repo (GitHub) - https://github.com/langfuse/langfuse
- R17 - promptfoo (open-source evals + red teaming) repo (GitHub) - https://github.com/promptfoo/promptfoo
- R18 - LangGraph (stateful workflow/agent orchestration) repo (GitHub) - https://github.com/langchain-ai/langgraph
- R19 - CrewAI (multi-agent orchestration) repo + MIT license (GitHub) - https://github.com/crewAIInc/crewAI
- R20 - AutoGen (multi-agent framework) repo (GitHub) - https://github.com/microsoft/autogen
- R21 - OpenHands (coding agent platform) repo (GitHub) - https://github.com/OpenHands/OpenHands
- R22 - Aider (terminal pair-programmer) repo (GitHub) - https://github.com/Aider-AI/aider
- R23 - Cline (IDE agent) repo showing stars/forks (GitHub) - https://github.com/cline/cline
- R24 - Continue (AI checks on every PR) repo (GitHub) - https://github.com/continuedev/continue
- R25 - AGENTS.md standard (GitHub) - https://github.com/agentsmd/agents.md
- R26 - Vale (prose style linter) (GitHub) - https://github.com/errata-ai/vale
- R27 - Proselint (prose linter) (GitHub) - https://github.com/amperser/proselint
- R28 - alex (inclusive language linter) (GitHub) - https://github.com/get-alex/alex
- R29 - Presidio (PII detection/redaction) (GitHub) - https://github.com/microsoft/presidio
- R30 - NeMo Guardrails (programmable guardrails) (GitHub) - https://github.com/NVIDIA-NeMo/Guardrails
- R31 - Guardrails AI (I/O validation + structure) (GitHub) - https://github.com/guardrails-ai/guardrails
- R32 - Gitleaks (secret scanning) (GitHub) - https://github.com/gitleaks/gitleaks
- R33 - OSV-Scanner (dependency vuln scanning) (GitHub) - https://github.com/google/osv-scanner
- R34 - Trivy (container/fs/IaC scanning) (GitHub) - https://github.com/aquasecurity/trivy
- R35 - Opengrep (open SAST fork) repo (GitHub) - https://github.com/opengrep/opengrep

- R36 - Meilisearch (open source search engine) repo (GitHub) - https://github.com/meilisearch/meilisearch
- R37 - Qdrant (vector DB) repo (GitHub) - https://github.com/qdrant/qdrant
- R38 - changedetection.io (web change/price monitoring) (GitHub) - https://github.com/dgtlmoon/changedetection.io
- R39 - Huginn (IFTTT-like agents) (GitHub) - https://github.com/huginn/huginn
- R40 - MkDocs Material (docs) (GitHub) - https://github.com/squidfunk/mkdocs-material
- R41 - Docusaurus (docs) (GitHub) - https://github.com/facebook/docusaurus
- R42 - Reddit thread: OpenWebUI vs LibreChat (community comparison) (Reddit) - https://www.reddit.com/r/LocalLLaMA/comments/1l0bc5j/openwebui_vs_librechat/
- R43 - Reddit thread: awesome MCP servers (2000+ servers hype signal) (Reddit) - https://www.reddit.com/r/LocalLLaMA/comments/1jromm0/found_an_awesome_repo_listing_more_than_2000_mcp/
- R44 - Instagram: Open WebUI install hype signal (Instagram) - https://www.instagram.com/reel/DJPi9rwStEN/
- R45 - Instagram: Understanding MCP hype signal (Instagram) - https://www.instagram.com/p/DTXccLMAcxi/
- R46 - TechRadar: Open WebUI Direct Connection vulnerability (patch guidance) (News) - https://www.techradar.com/pro/security/this-webui-vulnerability-allows-remote-code-execution-heres-how-to-stay-safe